

Status: accepted · **Date:** 2026-07-24 · **Author:** Will Hooper

ABRA simulates Champions battles with `engine/medicham2-browser.js` , a rules engine written by hand and corrected incrementally. On 2026-07-24 a single working session found the following defects in it, every one of them silent — the engine returned a number, it was simply the wrong number:

Defect	Effect
Status moves applied a uniformly random status	Thunder Wave burned a third of the time
Only Fake Out could flinch	Rock Slide's 30%, Iron Head's 30% and 31 others were inert
No status immunities	Fire types were burned, Electric types paralysed
Priority was a hand-typed table of 18 moves	All 14 negative-priority moves resolved at 0; Trick Room (−7) went at normal speed
Flinch was never cleared at end of turn	A flinch from a slower attacker stole the target's <i>next</i> turn
Intimidate was an unconditional −1	Blocked by 10 abilities; reversed by Defiant (+2) and Competitive (+2 SpA) — the sign was wrong on the format's most-used ability
No powder immunity	Spore hit Grass types
Prankster hit Dark types	Illegal since Gen 7

Five of these were found because the project owner happened to mention them in conversation. That is not a defect-detection process. The ones nobody mentioned are still there.

Measured impact of the corrections that were made: on 120 real matchups from the store, mean change in P(win) was **4.35 points**, maximum **24.2 points**, and the favoured side **flipped in 9.2%** of matchups. These are not cosmetic differences.

`data/mods/champions/` **exists in the Showdown master branch** and implements the format exactly:

<code>abilities.ts</code>	2.5 KB	the Champions-specific abilities
<code>conditions.ts</code>	1.8 KB	paralysis / sleep / freeze mechanics
<code>formats-data.ts</code>	75.4 KB	legality and tiering
<code>items.ts</code>	16.3 KB	mega stones
<code>learnsets.ts</code>	278.5 KB	what every species can learn
<code>moves.ts</code>	20.3 KB	move changes
<code>rulesets.ts</code>	1.9 KB	the SP system
<code>scripts.ts</code>	21.3 KB	stat and damage calculation

Three constants this project had carried as **unsourced inline comments** were checked against `conditions.ts` and all three are exactly right:

Mechanic	Asserted in our engine	Champions source	Our measurement (7,948 raw logs)
Full paralysis	12.5%	<code>randomChance(1, 8)</code>	13.8%, CI [11.9, 16.0]
Sleep, wake on turn 2	33%	<code>sample([2, 3, 3])</code>	35.3%, CI [31.5, 39.2]
Freeze thaw per attempt	25%, forced at 3	<code>randomChance(1, 4)</code> , <code>startTime</code> <code>= 3</code>	31.6%, CI [23.3, 41.4]

They were right, but they were right by luck rather than by sourcing — nothing in the repository said where they came from, and standard **S8 (measured, never asserted)** was being violated the whole time. `conditions.ts` also carries `move.flags['defrost']` , the auto-thaw rule, which our engine does not implement at all.

The npm package `pokemon-showdown@0.11.10` does **not** contain the mod; it is present only in the master branch. That is why an earlier check of `@pkmn/sim` (201 formats, none matching) wrongly concluded no Champions engine existed.

Built from a master checkout, the simulator exposes **14 Champions formats**, including `gen9championsvgc2026regmb` - the exact format of every replay in the store.

Adopt `data/mods/champions/` **as the authoritative rules engine**. Drive it through the Showdown simulator API rather than continuing to maintain a parallel implementation.

This is standard **S1 (single source of truth)** applied at the largest scale in the project. We have been maintaining a second, worse implementation of rules that already have an authoritative one, and paying for it in silent wrongness.

Both engines were benchmarked on the same machine, single core, running complete battles to a winner.

	battles/sec/core	ms per battle
Official Champions sim (<code>gen9championsvgc2026regmb</code>)	29	34.9
Our hand-written engine	3,401	0.29

The official simulator is **117x slower**. That number decides the architecture, and it decides it cleanly, because the two uses have completely different budgets:

CORRECTED 2026-08-06 (3.59.0). The 117x does not reproduce. The measured ratio is 24.9x, and the original figures above are left in place because a prior conclusion is never silently rewritten. Re-measured on this machine, both engines on the same four teams (derived from the store, not typed), 8-second runs, 60-turn cap: `` turns/sec battles/sec MEDICHAM 13,041 217 champions_sim 523 28 ratio 24.9x 7.7x ` \*\* turns/sec is the comparable unit and battles/sec is not.\*\**** The two engines were driven differently – MEDICHAM to its 60-turn cap, Showdown with `choose('default')` to a natural end – so a "battle" is not the same quantity of work on the two sides and the 7.7x is not a like-for-like ratio. The honest statement of the gap is *24.9x****. ****This** ADR's architectural decision was justified with a number that is not true******, and that has to be said plainly. The decision itself survives the correction – a 24.9x gap still rules out live browser simulation, and the offline budgets below still finish in minutes – but every workload row in the next table was computed at 117x and is therefore pessimistic about the official simulator by roughly 4.7x. Read the rows as an ordering, not as durations. Two other readings of the same pair are on record and neither is this one: ROADMAP #61 measured MEDICHAM at ****1,606**** battles/sec against the 3,401 here, and `engine/champions_sim.js`'s header comment still states the 117x. The instrument, not the engine, is what moved most – nothing has ever ratcheted engine speed, which is why a gap this size went unnoticed for two weeks.

Workload	Official, 1 core	Official, 8 cores	Ours, 1 core
One matchup, 100 rollouts (a browser click)	3.4 s	0.4 s	0.03 s
One matchup, 400 rollouts (site quality)	13.8 s	1.7 s	0.1 s
Backtest: 927 clean games x 100 rollouts	53 min	7 min	27 s
Matchup matrix: 50 archetypes squared x 200	4.8 hr	36 min	2 min
Training set: 1M labelled positions	9.6 hr	1.2 hr	5 min

Offline: comfortably fast enough. Every batch job finishes in minutes to a couple of hours, and these are jobs that run when the data refreshes, not per request. 117x slower does not matter when the job was going to take two minutes.

Live in the browser: not viable, and not needed. CHOMP's budget is 50 ms. The official simulator manages 1.5 battles in 50 ms; our engine manages 170. No amount of tuning closes a 117x gap.

3.59.0: *the gap is 24.9x on the comparable unit, not 117x, and the conclusion is unchanged — no amount of tuning closes a 24.9x gap either.*

The resolution is that **the browser should not be simulating at all**. Precompute matchup values offline with the official simulator and ship the table. That is strictly better than today on both axes at once: the numbers become correct *and* a lookup is faster than 170 rollouts of a wrong engine. The speed of our hand-written engine was only ever needed because we were recomputing, at request time, something that does not change between requests.

Gained

- Every rule is correct by construction, including the ones nobody has thought to check.
- Champions-specific mechanics come from the format's own definition, not from inference.
- Rule changes arrive by updating a dependency instead of by someone noticing a bug.
- Damage, accuracy, priority, immunities, abilities and items stop being our problem.

Given up

- Live simulation in the browser. Measured at 117x slower, this is gone and is not coming back. It is replaced by precomputed tables, which is a better answer than the one it replaces.
- Interactive exploration of arbitrary matchups. Anything outside the precomputed set now costs 3.4 s rather than 30 ms, or needs a server.
- The current engine's damage output is validated to within 5% of `@smogon/calc` across 31 scenarios. That golden master must keep passing against the new path before any switch (**S9**).

Risks

- The master branch lists Reg **M-A**; our stored games are Reg **M-B**. The mod may have moved since the data was collected, so format versioning has to be handled explicitly.
- Depending on an unreleased branch means pinning a commit, not a version number.

`engine/champions_sim.js` runs the official simulator on the real format; `engine/prior_player.js` ports our behaviour-clone policy into it. Getting to a valid comparison took **four** attempts, and the three failures are recorded because each produced a number that looked like a finding and was not:

Attempt	Mean diff	What it actually measured
1. Teams filled from the raw learnset	32.2 pts	The filler. Charizard got <i>Acrobatics</i> , <i>Aerial Ace</i> , <i>Air Cutter</i> , <i>Air Slash</i> . Only 1.6 of 4 moves are revealed per replay set, 68% have no item, 76% no ability - so the filler dominated.
2. Identical teams, different policies	23.7 pts	The policy. Ours samples behaviour-cloned priors; <code>RandomPlayerAI</code> picks uniformly.
3. "Policy ported"	32.2 pts	Nothing. <code>active.species</code> is undefined in a Showdown request, so the port silently fell through to uniform random on 100% of decisions while reporting itself as a prior sampler.
4. Species read from <code>request.side.pokemon[i].details</code>	31.1 pts	Valid: identical teams, identical policy, 81.4% of decisions sampled from priors.

Two structural checks confirm neither engine has a side bias, so the gap is not an artifact of who moves first. Identical team on both sides:

Mirror	Our engine	Official sim
venusaur	49.8%	48.8%
sneasler	48.2%	45.0%
blastoise	50.1%	55.0%

The result. With identical teams, an identical policy and no side bias, our hand-written engine and the official simulator disagree by **31.1 percentage points on average** across 8 real clean matchups, and the favoured side **flips in 3 of 8**. Individual gaps reach 79 points.

For scale: all of today's rule corrections together - random status, flinch, immunities, priority, Intimidate - moved our engine by 4.35 points. The remaining disagreement with the authoritative implementation is **seven times larger than everything we fixed**. Fixing this engine by hand was never going to converge.

Residual caveats, stated rather than buried. 8 matchups at 60 battles each gives roughly a +/-12 point interval per cell, so the per-matchup numbers are noisy even though the mean is not. And 18.6% of simulator decisions still fall back to uniform random when a sampled prior move is illegal that turn, where our engine falls back to its best available attack. That residual policy difference is real and is not enough to explain a 31-point gap.

1. Pin the Showdown master commit; vendor `data/mods/champions/` with provenance (**S4**).
2. **Port the policy.** Implement the prior-sampling policy as a Showdown `BattlePlayer` so both engines decide identically. Without this, no engine comparison is interpretable (see above).
3. Run the existing 31-scenario damage golden master against the simulator. If it disagrees with `@smogon/calc`, resolve that before going further.
4. ~~Benchmark~~ **done**: 29 vs 3,401 battles/sec/core. Architecture is precompute-offline. (**Corrected 3.59.0 — re-measured at 13,041 vs 523 turns/sec, a ratio of 24.9x rather than 117x. See the correction note under "The speed question, measured". The architecture call stands.**)
5. Build the offline job: official simulator produces the matchup table for the archetypes the site shows. Runs on a data refresh, not per request.
6. Ship the table. `medicham2-browser.js` becomes a lookup with the current engine as fallback only for matchups outside the table.
7. Keep the contract test, re-pointed: the fallback engine must agree with the simulator.
8. Retire the hand-maintained tables (`ACC` , `SPREAD` , `MEGA_ABIL` , `PRIO_CONDITIONAL`).

Keep fixing the hand-written engine. Rejected. Today's session is the evidence: eight defects in one sitting, five surfaced by conversation rather than by any test, on an engine that had already been reviewed. The defect rate is not falling, and there is no reason to expect it to, because the specification it is being written against is not written down anywhere we control.